

---

# AchewGer-Zero: Reinforcement Learning in GuanDan Game via Policy Gradient and Reward Shaping

---

Jihao Huang<sup>1</sup> Kaiwen Gao<sup>2</sup>

## Abstract

GuanDan is a widely popular Chinese poker game that presents significant challenges to reinforcement learning due to its astronomical action space and intricate cooperative dynamics. In this paper, we present Achewger-Zero, an autonomous agent trained using the Proximal Policy Optimization (PPO) algorithm. To address the combinatorial explosion of card subsets, we employ a heuristic action generator that narrows the search space to strategically viable candidates. We propose a sophisticated feature encoding scheme that explicitly captures structural integrity and utilizes a covering-card-aware feasibility model. Furthermore, we implement a customized reward function that incorporates a speed bonus to incentivize early finishing, combined with Generalized Advantage Estimation (GAE) to effectively mitigate the challenges of long horizons and delayed rewards. Additionally, we utilize a mixed training regime where the agent co-evolves against a curriculum of rule-based baselines, random opponents, and historical self-checkpoints to ensure robust policy generalization. Our results demonstrate that Achewger-Zero successfully develops high-level tactical awareness, including strategic bomb preservation and efficient level-card utilization. Notably, the agent achieved superior performance and competitive rankings in the Botzone leaderboard matches.

## 1. Introduction

GuanDan is a highly strategic and cooperative trick-taking card game played with two standard decks of cards. Originating in China, it has recently gained immense popularity globally. The game’s complexity is driven by several unique mechanics: the inclusion of 108 cards, a dynamic “Wild Card” system, and a complex hierarchy of combinations.

As illustrated in Figure 1, from a reinforcement learning perspective, GuanDan presents an unprecedented state and action space complexity. Specifically, its information set count is estimated at  $10^{123}$  (Figure 1), which is orders of magnitude larger than that of Texas Holdem or DouDiZhu (Zha et al., 2021) (Lu et al., 2023). Furthermore, the number of legal actions in GuanDan can exceed  $10^6$  in certain states (Figure 1), far surpassing the branching factors seen in traditional card games. This astronomical complexity makes standard RL exploration and gradient-based optimization highly inefficient.

Despite its strategic depth, developing autonomous agents for GuanDan faces two primary hurdles. First, the combinatorial explosion of legal actions makes standard RL exploration highly inefficient. Second, successful play requires a delicate balance between individual card shedding and cooperative resource management. Traditional heuristic-based agents often fail to adapt to the diverse strategies encountered in competitive play.

In this paper, we present **Achewger-Zero**, an end-to-end autonomous agent trained via the Proximal Policy Optimization (PPO) algorithm (Schulman et al., 2017) (Lu et al., 2023). Our system integrates several key innovations:

- **Action Pruning via Heuristic Generation:** We implement a specialized action generator that filters the massive legal action set into a strategically viable candidate distribution.
- **Structure-Aware Feature Encoding:** We propose a novel state-action encoding scheme that explicitly monitors “structural integrity.”
- **Mixed Training Regime:** To ensure robustness, our agent co-evolves against a curriculum of rule-based baselines and historical checkpoints.

Empirical results demonstrate that Achewger-Zero develops sophisticated tactical behaviors. Notably, the agent achieved

---

<sup>1</sup>Yuanpei College, Peking University, Beijing, China

<sup>2</sup>School of Electronics Engineering and Computer Science, Peking University, Beijing, China. Correspondence to: Jihao Huang <jackhuang040828@outlook.com>, Kaiwen Gao <2300012977@stu.pku.edu.cn>.

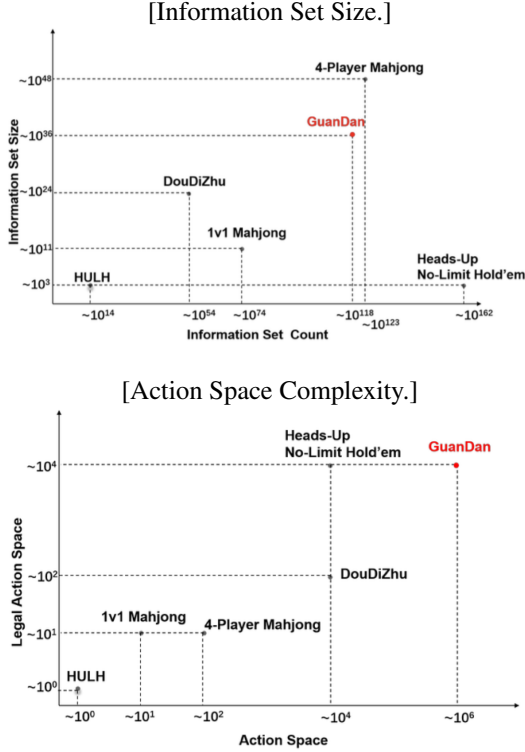


Figure 1. Comparison of game complexity. GuanDan (top of each chart) exhibits much higher state and action space dimensions than DouDiZhu or Mahjong.

superior performance in the Botzone leaderboard matches, demonstrating its competitive edge.

## 2. Methods

The core of AcheWger-Zero lies in its ability to abstract the high-dimensional game state of GuanDan into a structured representation that facilitates reinforcement learning. We partition our methodological framework into three components: feature encoding, action generation, and the mixed training strategy.

### 2.1. Feature Engineering

Effective state representation is critical in GuanDan due to the dual-deck nature and the presence of level-cards (wildcards). Our `FeatureEncoder` transforms the environment’s observation into a 68-dimensional state vector and a 69-dimensional action vector.

#### 2.1.1. STATE REPRESENTATION

The objective of the state encoder is to compress the partially observable environment into a fixed-length numerical representation  $\mathbf{s} \in \mathbb{R}^{68}$ . Unlike simpler card games, Guan-

Dan requires the agent to track not only its own hand but also the dynamic shifting of the “Wildcard” (level-card) and the strategic distribution of cards among other players.

As detailed in Table 1, our encoding scheme partitions the 68-dimensional vector into four primary functional modules:

1. **Private Information:** Indices 0–14 encode the current player’s hand. This is a count-based vector where each element represents the frequency of a rank (from 2 to A, including Jokers).
2. **Global Context:** Indices 15–30 provide the environment’s metadata, including a one-hot encoding of the current level rank and the player’s relative seat position.
3. **Opponent Modeling:** To facilitate endgame awareness, indices 31–39 explicitly track the number of cards remaining for the other three players. This allows the model to shift from a value-oriented strategy to a defensive, "bomb-preserving" strategy when an opponent is close to finishing.
4. **Historical Memory:** Indices 40–54 maintain a global count of all cards played throughout the match. Given that GuanDan uses two decks (108 cards), tracking the exhaustion of high-rank cards and jokers is paramount for calculating the probability of hidden bombs.

Table 1. Detailed dimensions and mapping of the 68-dimensional state vector  $\mathbf{s}$ .

| Index Range | Feature Description                        | Dim |
|-------------|--------------------------------------------|-----|
| 0 – 14      | Current hand card counts (2–A, jokers)     | 15  |
| 15 – 27     | Current level rank (One-hot A–K)           | 13  |
| 28 – 30     | Player’s relative position (One-hot)       | 3   |
| 31 – 33     | Cards remaining: Player 1 (Next)           | 3   |
| 34 – 36     | Cards remaining: Player 2 (Partner)        | 3   |
| 37 – 39     | Cards remaining: Player 3 (Previous)       | 3   |
| 40 – 54     | Global history: counts of all played cards | 15  |
| 55 – 67     | Level-specific features (Wildcard mapping) | 13  |

The final 13 dimensions (55–67) are dedicated to the "Heart-Level" wildcard system. By explicitly marking which cards can function as versatile substitutes, the agent can effectively evaluate complex combinations like straight-flushes that rely on wildcard assistance.

#### 2.1.2. ACTION REPRESENTATION

In GuanDan, an action is not merely defined by the cards played, but by the strategic intent (the “claim”) and its impact on the remaining hand. Our `FeatureEncoder` maps each candidate action generated by the heuristic engine into a 69-dimensional feature vector  $\mathbf{a} \in \mathbb{R}^{69}$ .

As summarized in Table 2, the action vector is designed to help the policy network evaluate the trade-off between immediate tactical gain and long-term resource preservation:

1. **Type and Rank (0–39):** We use a one-hot encoding for the 27 possible poker types defined in the environment (e.g., single, pair, straight, bomb). This is augmented by a one-hot encoding of the primary rank of the combination.
2. **Structural Integrity (40–65):** This is a critical innovation in Achewger-Zero. The encoder computes a “Break Penalty” by comparing the hand’s rank-multiplicity before and after the action. If a player uses cards that were part of a potential 4-of-a-kind (bomb) or a straight-flush, the corresponding indices (40–65) are activated. This explicitly signals to the PPO agent that the current action weakens its defensive or offensive potential.
3. **Length and Metadata (66–68):** These indices encode the number of cards in the action and specific flags for wildcard usage, allowing the model to distinguish between natural combinations and those assisted by the “Heart-Level” card.

Table 2. Detailed dimensions and mapping of the 69-dimensional action vector  $\mathbf{a}$ .

| Index Range | Feature Description                | Dim |
|-------------|------------------------------------|-----|
| 0 – 26      | Poker Type One-hot (27 types)      | 27  |
| 27 – 39     | Primary Rank One-hot (A–K)         | 13  |
| 40 – 52     | Break-Bomb Penalty (Rank-wise A–K) | 13  |
| 53 – 65     | Break-Straight-Flush Penalty (A–K) | 13  |
| 66          | Action Length (Normalized)         | 1   |
| 67 – 68     | Wildcard Usage Flags               | 2   |

## 2.2. Action Generation

GuanDan’s complexity is primarily characterized by its vast action space, where a single state can yield up to  $10^6$  legal combinations (see Figure 1). Directly optimizing a policy  $\pi(a|s)$  over such a large discrete space is computationally prohibitive. To address this, we propose a **Heuristic Action Generator**  $\mathcal{G}(s)$ , which prunes the raw action space  $\mathcal{A}$  into a candidate set  $\mathcal{C} = \{a_1, \dots, a_k\}$  where  $k \ll |\mathcal{A}|$ .

### 2.2.1. HIERARCHICAL PRUNING WITH PRIORITY MASKING

The generator employs a hierarchical logic to identify strategically dominant combinations. A key innovation is the **Structural Protection Mask**  $\mathbf{m} \in \{0, 1\}^N$  (where

$N = 108$ ), which identifies cards belonging to high-value patterns.

As implemented in `action_generator.py`, we prioritize the identification of Straight Flushes ( $SF$ ) due to their dual role as powerful bombs and high-scoring sequences. For each suit  $s \in \mathcal{S}$  and rank sequence  $\mathcal{R} \in \mathcal{V}_{SF}$ , we define the mask as:

$$m_i = \begin{cases} 1 & \text{if card } i \in \{c \in \text{Hand} \mid \text{Suit}(c) = s, \text{Rank}(c) \in \mathcal{R}\} \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

By masking these critical resources, the generator prevents lower-tier patterns (e.g., standard bombs or triples) from consuming cards that constitute an  $SF$  unless the alternative formation is of higher utility.

### 2.2.2. CANDIDATE SEARCH AND WILDCARD SUBSTITUTION

The generation of candidates follows the logic detailed in Algorithm 1. The generator handles the *Heart-Level* wildcard ( $w$ ) by dynamically substituting missing ranks in pattern templates. For a target pattern type  $T$  requiring rank set  $\mathcal{P}$ , an action  $a$  is valid if:

$$|\{r \in \mathcal{P} \mid r \notin \text{Hand}\}| \leq \mathcal{K}(w \in \text{Hand}) \quad (2)$$

---

#### Algorithm 1 Heuristic Action Generation and Pruning

---

**Input:** Observation  $obs$ , Hand  $\mathcal{H}$ , Last Move  $a_{last}$

**Output:** Candidate Set  $\mathcal{C}$

$\mathbf{m} \leftarrow \text{IdentifyStraightFlushes}(\mathcal{H})$  {Priority Masking}

$\mathcal{C} \leftarrow \{\text{Pass}\}$  if  $a_{last}$  exists

$\mathcal{B} \leftarrow \text{GenerateBombs}(\mathcal{H}, \mathbf{m})$  {Protect masked cards}

**if**  $a_{last}$  is bomb **then**

$\mathcal{C} \leftarrow \mathcal{C} \cup \{a \in \mathcal{B} \mid a \succ a_{last}\}$

**else**

$\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{B}$

**for each type**  $T \in \text{NormalTypes}$  **do**

$\mathcal{P} \leftarrow \text{EnumeratePatterns}(\mathcal{H}, T, \mathbf{m})$

$\mathcal{P}_{wild} \leftarrow \text{ApplyWildcardSubstitution}(\mathcal{P}, w)$

$\mathcal{C} \leftarrow \mathcal{C} \cup \{a \in (\mathcal{P} \cup \mathcal{P}_{wild}) \mid a \succ a_{last}\}$

**end for**

**end if**

$\mathcal{C} \leftarrow \text{DeduplicateByClaim}(\mathcal{C})$

**return**  $\mathcal{C}$

---

### 2.2.3. RANKING AND DEDUPLICATION

To ensure the policy network  $\pi(a|s)$  focuses on strategic quality rather than card-level permutations, the generator performs **Semantic Deduplication**. Actions that share identical “claims” (strategic intent) but differ in internal card IDs are collapsed into a single candidate:

$$a_i \sim a_j \iff \text{Claim}(a_i) = \text{Claim}(a_j) \quad (3)$$

This final pruning step ensures that  $|\mathcal{C}| \approx 10 \sim 50$ , providing a dense and high-quality distribution for the PPO actor-critic update.

### 2.3. Training Strategy

To ensure that Achewger-Zero develops a robust and generalized policy, we implemented a training pipeline centered around Proximal Policy Optimization (PPO) and a dynamic opponent curriculum.

#### 2.3.1. MIXED OPPONENT TRAINING REGIME

Training a GuanDan agent via pure self-play often leads to "strategy collapse" or overfitting to a specific partner's playstyle (Silver & et al., 2017). As implemented in `train_ppo_mixed.py`, we introduce a **Mixed Opponent Sampling** strategy to diversify the agent's experience.

Let  $\mathcal{P}_{pool} = \{\pi_{curr}, \pi_{hist}, \pi_{rule}, \pi_{rand}\}$  be the set of potential opponent policies. At each update index  $k$ , an opponent policy  $\pi_{opp}$  is sampled according to a curriculum-based distribution  $P(\pi|k)$ :

$$\pi_{opp} \sim P(\pi|k, \mathcal{M}), \quad \mathcal{M} = \text{choose\_opponent\_mode}(k) \quad (4)$$

The detailed curriculum logic, which transitions from rule-based guidance to competitive self-play, is formalized in Algorithm 2.

---

#### Algorithm 2 Mixed Opponent Training Regime

---

**Input:** Update index  $k$ , Warm-up threshold  $T_{wm}$ , Checkpoint pool  $\mathcal{S}_{ckpt}$   
**Output:** Opponent Mode  $\mathcal{M}$   
**if**  $k < T_{wm}$  **then**  
 $\mathcal{M} \leftarrow \text{Sample}(\{\text{Rule} : 0.4, \text{Self} : 0.55, \text{Rand} : 0.05\})$   
**else**  
**if**  $\mathcal{S}_{ckpt}$  is empty **then**  
 $\mathcal{M} \leftarrow \text{Sample}(\{\text{Rule} : 0.1, \text{Self} : 0.85, \text{Rand} : 0.05\})$   
**else**  
 $\mathcal{M} \leftarrow \text{Sample}(\{\text{Rule} : 0.1, \text{Self} : 0.6, \text{Past} : 0.25, \text{Rand} : 0.05\})$   
**end if**  
**end if**  
**return**  $\mathcal{M}$

---

#### 2.3.2. PPO OPTIMIZATION AND REWARD SHAPING

We optimize the agent using an Actor-Critic architecture where the policy network  $\pi_\theta(a|s)$  and value network  $V_\phi(s)$  share partial features. The objective function is the clipped PPO loss (Schulman et al., 2017):

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[ \min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad (5)$$

where  $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ . To handle the high variance of GuanDan's long-horizon rewards, we utilize **Generalized Advantage Estimation (GAE)** (Schulman et al., 2015) to compute the advantage  $\hat{A}_t$ :

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \quad (6)$$

Furthermore, we incorporate a **Speed Bonus**  $R_{speed}$  into the terminal reward  $R_{term}$ . Given a reference time  $t_{ref}$  and the actual finish time  $T$ , the bonus is defined as:

$$R_{speed} = \alpha \cdot \max \left( 0, \frac{t_{ref} - T}{t_{ref}} \right) \quad (7)$$

where  $\alpha$  is a scaling factor (set to 0.5 in our experiments). This incentivizes the agent to prioritize finishing first rather than simply winning the trick-taking sequence.

## 3. Experiments and Results

In this section, we present a comprehensive evaluation of Achewger-Zero. We compare our proposed mixed-opponent curriculum (Run 2) against a pure self-play baseline (Run 1) to demonstrate the necessity of strategic diversification.

### 3.1. Experimental Setup

The training and evaluations were conducted on a high-performance server equipped with an NVIDIA GeForce RTX 4090 GPU. The environment follows the standard GuanDan rules as implemented on the Botzone platform. Hyperparameters were kept consistent across both runs: learning rate  $\eta = 10^{-4}$ , discount factor  $\gamma = 0.99$ , GAE parameter  $\lambda = 0.95$ , and a PPO clipping epsilon of 0.2.

### 3.2. Training Regimes and Hyperparameters

We compared two distinct training configurations:

- **Run 1 (Pure Self-play):** The agent trains exclusively against its current version.
- **Run 2 (Mixed Curriculum):** The agent utilizes a staged curriculum. As shown in our configuration, the agent first faces rule-based agents with a higher probability (30%) during warm-up, and later incorporates past checkpoints (15%) and random agents (5%) to maintain robustness.

### 3.3. Convergence and Reward Analysis

Figure 2 illustrates the average episode reward across 1,000 updates.

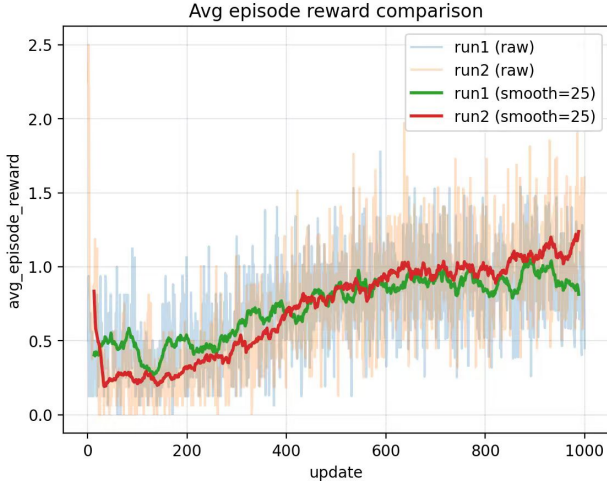


Figure 2. Average episode reward comparison. Run 2 (red) demonstrates more stable growth and higher terminal rewards compared to the raw oscillations in Run 1 (green).

Run 2 exhibits a smoother and more consistent upward trajectory. While Run 1 experiences significant reward collapses around update 200, the mixed-opponent strategy in Run 2 provides a more stable gradient signal, confirming that facing diverse opponents prevents the policy from converging to fragile local optima.

### 3.4. Performance Against Baselines

We evaluated the agents against a *RandomAgent* and our *RuleBasedAgent*.

#### 3.4.1. RESULTS AGAINST RANDOMAGENT

As shown in Figure 3, both runs quickly achieve high win rates against random play. Run 2 (orange) reaches a win rate near 0.95 earlier than Run 1 (blue). Notably, the rate of "+3 wins" (Big wins) in Run 2 grows steadily, indicating that the curriculum-trained agent is more proficient at clearing both team hands before the opponents.

#### 3.4.2. RESULTS AGAINST RULEBASEDAGENT

Figure 4 highlights the most significant disparity. Run 1 struggles to surpass a 0.5 win rate consistently, whereas Run 2 achieves a clear breakthrough, reaching nearly 0.6 win rate by update 1,000. This demonstrates that the mixed-opponent curriculum effectively teaches the agent how to dismantle structured human-like heuristics, which pure self-play fails to encounter.

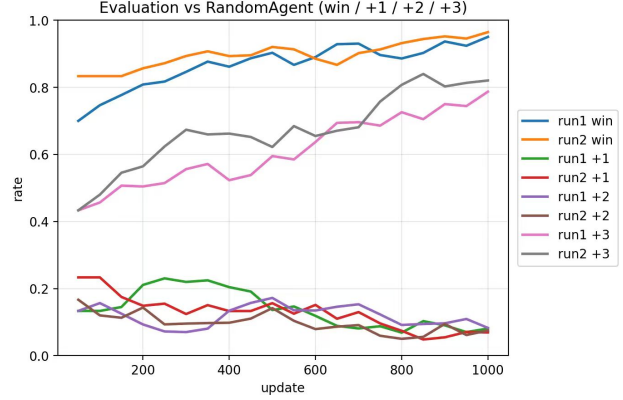


Figure 3. Win rate and scoring distribution vs. RandomAgent. Run 2 consistently outperforms Run 1 in both total wins and high-score (+3) outcomes.

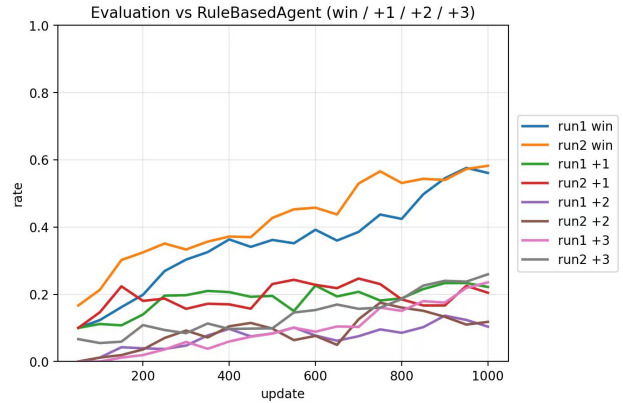


Figure 4. Evaluation vs. RuleBasedAgent. The curriculum-based Run 2 (orange) achieves higher win rates and breakthrough performance compared to the self-play baseline.

## 4. Evolution of Strategy and Training Methodology

The development of Achewger-Zero was an iterative process, evolving through three distinct stages of exploration to address the unique challenges of GuanDan.

### 4.1. Phase I: Foundations of PPO and Action Pruning

Our initial approach was inspired by state-of-the-art card game agents like *DouZero*. We adopted Proximal Policy Optimization (PPO) as our core algorithm, utilizing a self-play framework where four identical agents interacted within the environment to generate trajectories.

To handle the astronomical action space, we implemented

the `ActionGenerator` class for strategic pruning. This heuristic engine ensures 100% legal move generation by prioritizing high-value combinations like straight-flushes via a *Priority Masking* mechanism, which locks critical cards using a `straight_flush_mask`. For optimization, we utilized Generalized Advantage Estimation (GAE) to balance bias and variance in GuanDan’s long-horizon episodes. The PPO objective incorporated a policy loss with epsilon-clipping, a value loss for the critic, and an entropy regularization term to encourage strategic diversity and prevent premature convergence to local optima.

#### 4.2. Phase II: Addressing Mis-coordination via Reward Shaping

Initial Botzone evaluations revealed significant behavioral flaws. Most notably, the agents struggled with positional context, often failing to distinguish between leading a trick and responding to an ally. This led to "friendly fire," where the agent would use a high-value bomb to suppress its own teammate’s move.

To curb these irrational behaviors, we transitioned from sparse terminal rewards to dense reward shaping. We designed seven specific penalty and bonus rules, as summarized in Table 3. These included penalties for "using a cannon to shoot a mosquito" (Rule A) and "bombing a teammate" (Rule C). While this significantly suppressed undesirable actions, it introduced a new issue: the agent became overly "hesitant," frequently choosing to `pass` due to the high density of negative reinforcements.

Table 3. Reward Shaping Rules and Trigger Conditions.

| Rule   | Description                                 | Reward      |
|--------|---------------------------------------------|-------------|
| Rule A | Over-using bombs on weak enemy cards        | -0.2        |
| Rule B | Misusing level-cards for low-value types    | -0.5        |
| Rule C | Bombing a teammate’s move                   | -0.5        |
| Rule D | Passing on a teammate’s small single card   | -0.1        |
| Rule E | Breaking bombs for sequences (Success/Fail) | +0.1 / -0.4 |
| Rule F | Holding small cards at end-game             | -0.03/card  |
| Rule G | Not using the smallest effective bomb       | -0.2        |

#### 4.3. Phase III: Expert-Guided Curriculum and Speed Incentives

To overcome the "hesitation" observed in Phase II, we pivoted to a multi-stage curriculum. We developed a sophisticated `RuleBasedAgent` to act as an initial mentor. In the first 10% of training, the RL agent was guided by this rule-based logic to rapidly grasp basic GuanDan intuition.

Furthermore, inspired by peer insights from late December seminars, we refined our reward function to include a **Speed Bonus**. This mechanism rewards agents more for finishing the round earlier, aligning the AI’s objective with the

human goal of "finishing first". This combined approach of expert initialization and temporal incentives yielded the most competitive version of AcheWGer-Zero.

## 5. Discussion and Limitations

In this section, we provide a critical analysis of the methodologies employed in AcheWGer-Zero, focusing on the trade-offs between strategic efficiency and the potential biases introduced by our design choices.

### 5.1. Analysis of the Speed-Based Reward Mechanism

The introduction of a reward function that prioritizes finishing earlier (Speed Bonus) significantly altered the agent’s behavior.

- **Effectiveness:** This mechanism effectively aligned the RL agent’s objective with the core human strategy of “Qiang-You” (securing the top rank), leading to a higher win rate in competitive simulations.
- **Deficiencies:** However, we observed that an over-reliance on speed incentives can lead to a “shortsighted” strategy. In certain endgame scenarios, the agent might exhaust its only bomb to clear a small hand prematurely, failing to account for the risk of opponents regaining control with larger combinations. Balancing the terminal game reward with the temporal speed bonus remains a delicate optimization challenge.

### 5.2. Evaluation of Rule-Based Warm-up

Transitioning from explicit manual penalties to a 10% expert-guided warm-up phase was a pivotal turning point in our training regime.

- **Pros:** By mimicking the `RuleBasedAgent` in the early stages, the model bypassed the chaotic exploration phase where it previously struggled with basic concepts like not bombing its own teammate. This provided a stable foundation for subsequent PPO optimization.
- **Cons:** The primary risk of this approach is “policy anchoring.” If the initial rule-based mentor possesses inherent biases (e.g., being overly conservative with level-cards), the RL agent may inherit these sub-optimal patterns. Breaking away from these expert-imposed constraints to find truly novel strategies requires a sufficiently long self-play phase and a high entropy coefficient.

### 5.3. The Double-Edged Sword of Action Pruning

Our `ActionGenerator` reduces the branching factor from  $10^6$  to approximately 50, which is essential for conver-

gence but introduces structural limits:

- **Strategic Bottleneck:** The agent’s creativity is upper-bounded by the heuristic rules defined in the generator. For instance, if the generator does not propose an unconventional way to split a straight to form a defensive pair, the PPO policy will never explore that possibility.
- **Inflexibility:** The priority masking logic (e.g., always protecting straight-flushes) ensures stability but may prevent the discovery of elite-level plays where sacrificing a high-value sequence is the only path to victory.

#### 5.4. Coordination and Signaling

A notable limitation is the absence of explicit communication or signaling modeling. While our agent tracks played cards, it does not yet effectively “read” the partner’s intent or signal its own needs (e.g., signaling a lack of bombs). Developing a more sophisticated partner-awareness module remains a key area for future improvement (Zhao et al., 2024).

## 6. Conclusion

This paper presented **Achowger-Zero**, an autonomous agent addressing the high-dimensional and cooperative complexities of GuanDan. Our research demonstrates that combining heuristic-driven pruning with expert-guided curriculum learning effectively overcomes the “curse of dimensionality” inherent in action spaces exceeding  $10^6$ .

Key insights from our evolutionary approach include: (1) **Structural integrity features** are essential for long-term resource management and preventing sub-optimal decomposition of card combinations; (2) The **expert-guided warm-up** phase successfully bridges the gap between random exploration and strategic intuition, eliminating irrational behaviors like teammate suppression; (3) The **speed-based reward mechanism** aligns agent behavior with the human objective of finishing first.

Validation on **Botzone** confirms Achewger-Zero’s competitive performance and cooperative efficiency against diverse opponents. Future work will focus on modeling hidden signals and partner intent to further enhance collaborative prowess in high-stakes environments (Zhao et al., 2024) (Lowe & et al., 2017).

## Author Contributions

The development of **Achowger-Zero** was a collaborative effort with the following specific contributions:

- **Jihao Huang:** Designed and implemented the `ActionGenerator` for legal move pruning. He

developed the `RuleBasedAgent`, a sophisticated heuristic baseline used for expert-guided curriculum learning and strategic benchmarking. Furthermore, he designed the 68-dimensional and 69-dimensional feature encoding schemes and established the PPO training pipeline.

- **Kaiwen Gao:** Architected the comprehensive reward system, including the seven penalty rules and the speed-based incentive mechanism. He was responsible for the Botzone platform interface implementation and system integration. Additionally, he led the preparation of the mid-term report and the final research paper writing.

## References

- Lowe, R. and et al. Multi-agent actor-critic for mixed cooperative-competitive environments. *arXiv preprint arXiv:1706.02275*, 2017. URL <https://arxiv.org/abs/1706.02275>.
- Lu, Y., Zhao, J., Zhao, Y., Zhou, W., and Li, H. Danzero: Mastering guandan game with reinforcement learning. In *IEEE Conference on Games (CoG)*, pp. 1–8. IEEE, 2023. doi: 10.1109/COG57401.2023.10333179. URL <https://doi.org/10.1109/COG57401.2023.10333179>.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015. URL <https://arxiv.org/abs/1506.02438>.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Silver, D. and et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017. URL <https://arxiv.org/abs/1712.01815>.
- Zha, D., Xie, J., Ma, W., Zhang, S., Lian, X., Hu, X., and Liu, J. Douzero: Mastering doudizhu with self-play deep reinforcement learning. *arXiv preprint arXiv:2106.06135*, 2021. URL <https://arxiv.org/abs/2106.06135>.
- Zhao, Y., Lu, Y., Zhao, J., Zhou, W., and Li, H. Danzero+: Dominating the guandan game through reinforcement learning. *IEEE Transactions on Games*, 16(4):914–926, 2024. doi: 10.1109/TG.2024.3422396. URL <https://doi.org/10.1109/TG.2024.3422396>.